Semester Research Project Proposal

Project title: Retrieved but Ignored: Where LLMs Lose Code Reuse When Refactoring (Project 10: Code Reuse / Refactoring with AI)

Student name(s): Evgenii Lialin, Roman Vikhman

Course: Advanced Techniques in Information Retrieval

Supervisor: Marina Litvak

Semester: Fall 2026

1 Problem Statement and Introduction

Problem: Large language models (LLMs) used as coding assistants often re-implement logic that the repository already provides instead of calling the existing function. When such a model is asked to refactor a function, it may keep a duplicated block of code, or even write a new helper, although an equivalent helper already exists in the project.

Example: in a Python library, the function parse() contains the lines that strip and normalise an input line, while the module already has a helper _strip(line) that does exactly this. A good refactoring replaces the duplicated lines with a call to _strip(line). A model that does not know about the helper, does not find it, or sees it but ignores it, leaves the duplication in place.

Motivation: Duplicated code passes the tests, so the problem is invisible to test-based evaluation, but it is a long-term maintenance cost: every bug fix must be made in several places. Recent studies show that AI-generated pull requests contain more redundant code than human ones and that reviewers do not notice it. Developers and teams using AI coding assistants, and the researchers building retrieval-augmented code assistants, would benefit from knowing why reuse fails.

Retrieval-augmented code assistants assume that if the right code is retrieved, the model will use it. If the model instead ignores retrieved code, improving retrievers will not solve the problem, and effort should go into instructions, prompting or training instead.

Objective: The project will build a controlled benchmark of reuse-refactoring tasks in Python and measure where reuse opportunities are lost: (1) the existing helper is not retrieved into the context, (2) it is in the context but the model ignores it, or (3) the model calls it but the edit breaks the tests. We will vary the context given to the model and the instruction, for several open LLMs.

Current status: the benchmark (129 tasks from 12 repositories), the evaluation harness and the main runs for four open models are complete, and a first paper draft exists. The remaining part of the semester is planned for extensions (dense retrieval, more realistic tasks), error analysis and the final report.

2 Related Work

Three lines of work are relevant. (1) The reuse problem: RepoExec measures how often generated code invokes the available repository dependencies (Dependency Invocation Rate) and finds that models often re-implement them; an empirical study of AI-generated pull requests finds more redundancy and less reuse than in human code. (2) Refactoring benchmarks (RefactorBench, SWE-Refactor, MANTRA, CodeTaste) evaluate whether LLM refactorings preserve behaviour or match human choices, but none measures reuse of

existing code. (3) Retrieval for code (CodeRAG, CoRet, ReflectCode, Think-Search-Patch) improves what is retrieved from the repository, assuming the model will use it. Work on long contexts and instruction following (Lost in the Middle, CodeIF) shows that models use context unevenly and follow instructions inconsistently, which motivates our position and instruction conditions.

Reference	Approach and main finding	Limitation or relevance to this project
Hai et al., 2025 (RepoExec)	Repository-level generation benchmark; introduces the Dependency Invocation Rate; pretrained models often re-implement dependencies.	We adopt the invocation metric, but for refactoring, where the duplicate already exists and must be recognised.
Huang et al., 2026	AI-generated pull requests disregard reuse opportunities and are more redundant than human ones; reviewers do not notice.	Motivation: shows the problem in real development, but not where in the pipeline reuse is lost.
Gautam et al., 2025 (RefactorBench); Xu et al., 2026 (SWE-Refactor); Thillen et al., 2026 (CodeTaste)	Benchmarks of LLM refactoring: task completion, behaviour preservation, agreement with human refactorings.	Measure correctness or human alignment, not reuse; SWE-Refactor is Java-only.
Zhang et al., 2025 (CodeRAG); Fehr et al., 2025 (CoRet)	Better retrieval of repository code for completion and editing improves downstream results.	Assume retrieved code is used; we test this assumption by guaranteeing the helper is in context.
Liu et al., 2024 (Lost in the Middle); Yan et al., 2025 (CodeIF)	Models use information unevenly depending on its position; instruction following in code generation varies widely.	Basis for our position ablation and for comparing a generic with an explicit reuse instruction.

Research gap: It is unknown at which stage reuse is lost when an LLM refactors code: retrieval, the model's decision to call the helper, or the edit itself. Existing benchmarks either do not measure reuse or do not control what the model sees. Our benchmark has a known reference for every task and controls the context, so each failure can be attributed to exactly one stage (the reuse funnel).

3 Research Questions or Hypotheses

Main research question: When an LLM refactors a function that duplicates an existing repository helper, at which stage is the reuse opportunity lost: the helper is not retrieved, it is retrieved but ignored, or it is called but the edit breaks the tests?

Optional additional question(s): RQ2: Does an explicit reuse instruction increase reuse, and at what cost in correctness? RQ3: Do newer and larger models reuse more? RQ4: Does reuse depend on surface cues: the helper's position among the retrieved snippets, and its name (memorisation of the public repositories versus naming cues)?

Hypothesis or hypotheses, if applicable: H1: The model's decision, not retrieval, is the main bottleneck: even with the helper guaranteed in the context, models will often ignore it (justification: RepoExec observes re-implementation even when dependencies are available). H2: An explicit reuse instruction increases reuse in every context condition. H3: Reuse depends on the helper's position in the context (Lost in the Middle). H4: Showing the helper under a neutral name reduces reuse, but direct memorisation of the original code is rare. These are hypotheses to test; improvements are not assumed.

4 Study Design

Dataset

- **Dataset name and source:** Our own benchmark, built automatically from 12 open-source Python libraries at pinned commits (boltons, cachetools, click, mistune, parso, pycparser, pyflakes, Pygments, python-tabulate, sqlparse, tomlkit, toolz).
- **Task, language, and domain:** Reuse-oriented refactoring of Python functions in general-purpose libraries. Each task is a function F that duplicates the logic of an existing helper H; the ideal refactoring replaces the duplicate with a call to H.
- **Size and unit of analysis:** 129 tasks (36 with the helper in another module); helpers have 2–30 lines (median 7). The unit of analysis is one model output per task, condition and instruction (1,032 outputs per model for the main conditions).
- **Labels or relevance judgments:** No manual labels are needed: tasks are created by inlining H into one of its callers, so the original F (which calls H) is a known reference. Correctness is judged by the repository's own test suite; reuse is detected by AST analysis of the output (calls to H or a near-identical copy of H).
- **Training, development, and test split:** No training is planned, so there is no training split. All 129 tasks are used for evaluation. Prompt wording and output-parsing rules were fixed on a pilot (5 tasks from boltons) and a smoke run before the main runs.
- **Preprocessing:** For each repository: index all functions, find statement-level calls F → H, inline H's body at the call site (argument binding, renaming locals, early returns), and keep the task only if the call site is covered by tests and the full test suite still passes. Overridable methods are skipped.
- **Access, licensing, or privacy considerations:** All repositories are public with permissive open-source licences (MIT or BSD); no personal data is involved. Since the repositories predate the models, contamination is checked by a renaming experiment.

Data creation is fully automatic (about 360 candidate tasks were found; a seeded sample of up to 12 per repository, plus all cross-module tasks, was kept). A small manual inspection of outputs is planned for error analysis (see Evaluation).

Methods and Models

Baseline: The NONE condition: the model sees only the target function and the module's imports, with a generic refactoring instruction. It shows how often models reuse a helper they are not shown.

Proposed approach: We vary two factors. Context: NONE; BM25 (top-5 repository functions retrieved with F as the query); ORACLE (H guaranteed among 4 BM25 distractors at a random position, an upper bound for retrieval); FILE (the whole module, a realistic editor view). Instruction: generic ("improve readability and maintainability") or reuse (adds "where possible, reuse existing functions from the project"). Every output is evaluated with the reuse funnel. Two control experiments: the helper first vs. last among the oracle snippets, and the helper renamed to a neutral name in the prompt.

Models and tools: Qwen2.5-Coder-7B-Instruct and DeepSeek-Coder-6.7B-Instruct (bf16, Hugging Face Transformers); Qwen3-Coder-30B-A3B-Instruct and Qwen3.8-27B (4-bit, vLLM). Python, pytest, our own BM25 implementation, AST analysis. One NVIDIA A5000 GPU (24 GB).

Condition	Method or model	Input or configuration	Purpose
Baseline	4 open LLMs	NONE context; generic and reuse instruction	Reference performance
Experiment 1	Same models	BM25 top-5 context	Test whether retrieved context increases reuse
Experiment 2	Same models	ORACLE context (helper always present)	Test whether the model's decision is the bottleneck (H1)
Experiment 3	Same models	FILE context (whole module)	Test a realistic editor-like view
Experiment 4	Same models	ORACLE with helper first vs. last	Test the effect of position (H3)
Experiment 5	Qwen3-Coder-30B, Qwen3.8-27B	Helper renamed to a neutral name in the prompt	Separate memorisation from naming cues (H4)

Controlled factors: Same 129 tasks, prompt template, output format and token limit (2,048 new tokens) for all models; greedy decoding; the same context budget (5 snippets) for BM25 and ORACLE; the same distractors in all ORACLE variants; the same test suites and evaluation code.

Development decisions: There is nothing to train or tune on the test tasks. The prompt was fixed after the pilot; fixes to the output parser found during the runs are applied to all outputs of all models, and their effect is reported.

Evaluation

- **Primary metric:** Helper reuse rate: the share of tasks in which the refactored function calls the helper (or a near-identical copy of it). This follows RepoExec's Dependency Invocation Rate and directly measures whether the reuse opportunity was taken.
- **Secondary metrics:** Validity (the repository tests pass and the edit is compliant: signature kept, no existing code redefined); the reuse funnel stages (not in context /

ignored / broken / success); BM25 Recall@5 and MRR of the helper; share of outputs that write a new helper; uses of the original helper name in the renaming experiment.

- **Comparison procedure:** All models and conditions are run on the same tasks, so comparisons are paired task by task (e.g. ORACLE vs. BM25, reuse vs. generic instruction, model vs. model).
- **Uncertainty or statistical analysis:** Exact McNemar tests on the discordant tasks for every paired comparison.
- **Error analysis:** The funnel assigns every failure to one stage. We will inspect a sample of outputs from each stage, in particular "ignored" cases with the helper in context, "broken" calls, and outputs that call a different repository function than the expected helper.
- **Success criteria:** H1 is supported if, in the ORACLE condition, the share of ignored helpers is much larger than the share BM25 fails to retrieve. A useful negative result would be that ORACLE reuse is close to 100%, which would mean retrieval is the real bottleneck.

5 Implementation and Reproducibility

The implementation is a Python package (reuse_bench) with separate steps: building tasks for a repository, generating model outputs for each condition, applying and testing the edits, and summarising the metrics. A single script (run.sh) runs each experiment on the GPU server, and the paper tables are generated from the stored results by one script, so every number can be reproduced.

The submission will include:

- Source code and environment requirements.
- Data access and preparation instructions.
- Model identifiers, configurations, and prompts, where applicable.
- Experiment and evaluation scripts.
- Results and instructions for reproducing them.

Estimated resources and costs: One NVIDIA A5000 (24 GB). About 4–5 hours per 7B model for the main conditions and about 2.5 hours for the renaming check; no API costs. Model weights are downloaded from Hugging Face.

6 Work Plan and Deliverables

Week(s)	Date	Deliverable	Description
1–3	6/11/26	Topic Presentation	Problem Statement, Introduction, Related Work
4	13/11/26	Study Presentation	Research Question or Hypothesis; Study Design: Dataset, Method, Models, Evaluation
1–4	13/11/26	Baseline implementation	Benchmark builder, 129 tasks, evaluation harness, NONE and BM25 runs (completed)
5–8	11/12/26	Main experiments	ORACLE, FILE, position and renaming runs for four models (completed); extensions: dense retrieval, paraphrased duplicates
9–11	1/1/27	Results and error analysis	Reuse funnel, statistical tests, manual analysis of failures
12–14	22/1/27	Final submission	Report, code, and presentation

7 Expected Contribution Risks and Scope

Expected contribution: A reproducible benchmark for reuse-oriented refactoring in Python with known references, an evaluation of four open LLMs, and the reuse funnel, which shows at which stage reuse is lost. A practical finding for tool builders: whether better retrieval or better instructions matter more.

Main risks and mitigation: (1) Contamination: the repositories are public and older than the models; mitigated by the renaming experiment. (2) Artificial tasks: inlined code is a near-copy of the helper; mitigated by paraphrasing the duplicates or mining real reuse commits as an extension. (3) Weak tests: passing tests do not prove equivalence; we keep only tasks whose call site is covered. (4) Limited GPU: large models are run in 4-bit quantisation.

Minimum achievable scope: The 129-task benchmark with two models, four context conditions and two instructions, analysed with the reuse funnel. This scope is already completed.

Optional extensions: Dense retrieval (e.g. CoRet-style), paraphrased duplicates, tasks mined from real commits, an agentic search condition, and a small LoRA fine-tune for reuse.

Limitations: Python only, 12 libraries, open models only, greedy decoding and a single run per prompt; duplicates are near-verbatim copies of the helper. Conclusions are restricted to this setting.

References

Du, J., et al. (2025). DependEval: Benchmarking LLMs for repository dependency understanding. Findings of ACL 2025. https://aclanthology.org/2025.findings-acl.373/

Fehr, F. J., et al. (2025). CoRet: Improved retriever for code editing. ACL 2025 (short papers). https://aclanthology.org/2025.acl-short.62/

Gautam, D., et al. (2025). RefactorBench: Evaluating stateful reasoning in language agents through code. ICLR 2025. https://arxiv.org/abs/2503.07832

Hai, N. L., et al. (2025). On the impacts of contexts on repository-level code generation (RepoExec). Findings of NAACL 2025. https://aclanthology.org/2025.findings-naacl.82/

Huang, H., et al. (2026). More code, less reuse: Investigating code quality and reviewer sentiment towards AI-generated pull requests. arXiv:2601.21276. https://arxiv.org/abs/2601.21276

Liu, N. F., et al. (2024). Lost in the middle: How language models use long contexts. TACL, 12. https://aclanthology.org/2024.tacl-1.9/

Liu, A., et al. (2025). Beyond function-level search: Repository-aware dual-encoder code retrieval with adversarial verification (ReflectCode). Findings of EMNLP 2025. https://aclanthology.org/2025.findings-emnlp.1147/

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4).

Thillen, A., et al. (2026). CodeTaste: Can LLMs generate human-level code refactorings? ICML 2026. https://arxiv.org/abs/2603.04177

Xiong, B., et al. (2025). Think-Search-Patch: A retrieval-augmented reasoning framework for repository-level code repair. EMNLP 2025 Industry Track. https://aclanthology.org/2025.emnlp-industry.109/

Xu, Y., et al. (2025). MANTRA: Enhancing automated method-level refactoring with contextual RAG and multi-agent LLM collaboration. arXiv:2503.14340. https://arxiv.org/abs/2503.14340

Xu, Y., Yang, J., & Chen, T.-H. (2026). SWE-Refactor: A repository-level benchmark for real-world LLM-based code refactoring. arXiv:2602.03712. https://arxiv.org/abs/2602.03712

Yan, K., et al. (2025). CodeIF: Benchmarking the instruction-following capabilities of large language models for code generation. ACL 2025 Industry Track. https://aclanthology.org/2025.acl-industry.89/

Zhang, S., et al. (2025). CodeRAG: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. EMNLP 2025. https://aclanthology.org/2025.emnlp-main.1187/

Models: Hui et al. (2024), Qwen2.5-Coder technical report, arXiv:2409.12186; Guo et al. (2024), DeepSeek-Coder, arXiv:2401.14196; Yang et al. (2025), Qwen3 technical report, arXiv:2505.09388; Kwon et al. (2023), vLLM, arXiv:2309.06180.